VidalStore — Arquitectura Cloud Native

Manual Técnico de Inspección, Métricas de RabbitMQ, Eventos y Reproceso DLQ (EP2)

Asignatura:	DSY1107 — Desarrollo Cloud Native I · 40% del ET (Evaluación Parcial 2)
Institución:	DUOC UC — Escuela de Informática y Telecomunicaciones
Equipo y Roles:	**Oscar Garrido:** vidalstore-admin (IE6, IE7, IE8, IE17, IE18) **David:** vidalstore-infra (IE2, IE9, IE10, IE11, IE14, IE19) **Iván:** vidalstore-eventos (IE3, IE4, IE5, IE12, IE13, IE15, IE16)
Enfoque de Oscar:	Microservicio administrativo NestJS 12 (puerto **3020**). Encapsula RabbitAdminService para auditoría en tiempo real con HTTP Management API (:15672), productor amqplib seguro y reproceso de mensajes muertos DLQ.
Infraestructura:	Clúster RabbitMQ 4.3 de 2 nodos (HA), PostgreSQL 18 (:5432) y Docker Compose unificado.
Estado de Cobertura:	**100% Tests Pasando en vidalstore-admin (60 Unitarios y 17 E2E con Vitest, Total: 77 Tests, CI listo)**

Alcance del Documento y Metas de la EP2

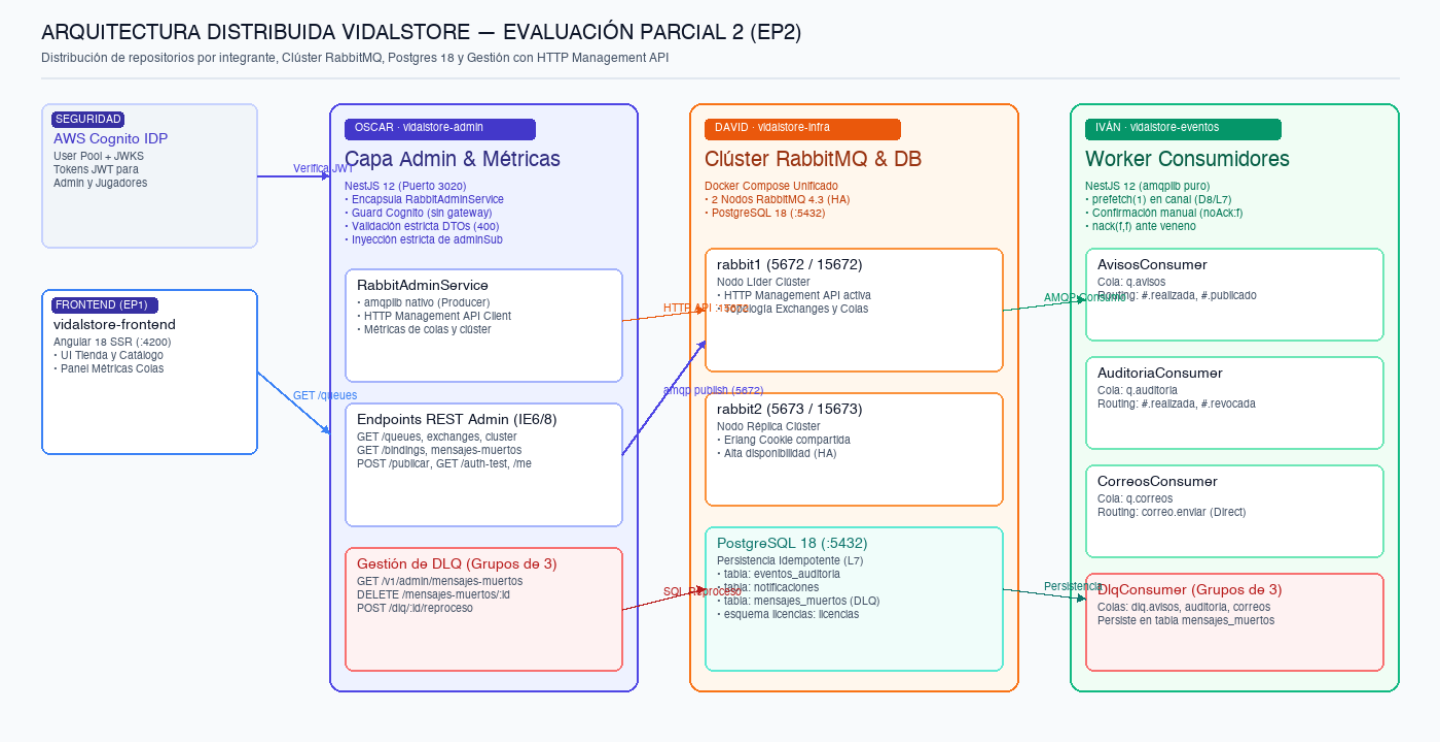
Este manual técnico constituye la referencia arquitectónica y operativa oficial para la **Evaluación Parcial 2 (EP2)**. A diferencia de la EP1 (enfocada en API Gateway y agregación sincrónica BFF), la EP2 establece una **Arquitectura Orientada a Eventos (EDA)** de alta resiliencia. En este ecosistema, **vidalstore-admin** no es únicamente un emisor de eventos, sino el nodo central de observabilidad, inspección y gestión administrativa del clúster de mensajería, auditando métricas de colas en tiempo real y habilitando el flujo de reprocesamiento de contingencia ante fallos.

OBJETIVOS DE EVALUACIÓN INDIVIDUAL Y GRUPAL

Rúbrica Oficial EP2: El microservicio vidalstore-admin liderado por Oscar responde directamente por los indicadores **IE6** (Endpoints REST con validaciones estrictas y DTOs), **IE7** (Seguridad Cognito JWT sin gateway perimetral), **IE8** (RabbitAdminService encapsulando Management API y amqplib), **IE17** (Demostración en vivo de operaciones admin) e **IE18** (Defensa técnica de métricas de colas, políticas de HA y gestión de DLQ).

1. Mapa del Ecosistema de Repositorios (EP2)

La EP2 divide el trabajo en repositorios desacoplados con responsabilidades e indicadores claramente delimitados:



Repositorio / Dueño	Tecnología	Puertos	Responsabilidad Técnica en la EP2
vidalstore-admin Oscar *(IE6, 7, 8, 17, 18)*	NestJS 12 + amqplib + Axios	**3020**	Capa administrativa y observabilidad. Encapsula **RabbitAdminService**, consulta HTTP Management API (:15672), publica con JWT y gestiona reproceso DLQ.
vidalstore-infra David *(IE2, 9, 10, 11, 14, 19)*	Docker Compose, RabbitMQ 4.3, Postgres 18	rabbit1: 5672/15672 rabbit2: 5673/15673 DB: 5432	Clúster de 2 nodos con Erlang cookie compartida, alta disponibilidad (HA), PostgreSQL 18 para persistencia y definición centralizada de shared/topologia.ts.
vidalstore-eventos Iván *(IE3, 4, 5, 12, 13, 15, 16)*	NestJS 12 + amqplib nativo	Worker interno	Microservicio worker consumidor. prefetch(1) (norma D8/L7), noAck: false, consumidores por dominio (Avisos, Auditoría, Correos, DlqConsumer).
vidalstore-frontend *Herencia EP1*	Angular 18 SSR	4200 / 4000	Panel de usuario y panel administrativo para visualizar el estado y métricas de las colas consultando GET /v1/admin/queues.

2. Misión Central de `vidalstore-admin` (IE6, IE7, IE8, IE17 e IE18)

En la arquitectura de la EP2, vidalstore-admin no se limita a ser un productor de eventos; constituye la **Capa de Inspección, Métricas y Gestión Administrativa de RabbitMQ**. Todo el acceso a la infraestructura del broker se encuentra estrictamente encapsulado en **RabbitAdminService**, prohibiendo que los controladores importen clientes AMQP o librerías HTTP de forma directa.

2.1 Interacción con la HTTP Management API de RabbitMQ (:15672)

RabbitMQ expone una API REST administrativa en el puerto 15672. RabbitAdminService se autentica mediante HTTP Basic Auth contra las credenciales del clúster (RABBITMQ_MANAGEMENT_URL, RABBITMQ_USER, RABBITMQ_PASSWORD) para inspeccionar la topología en tiempo real y mapear las respuestas a contratos DTO tipados:

- **/api/overview** y **/api/nodes**: Audita el estado de los nodos del clúster (rabbit1 y rabbit2), uptime, memoria y particiones de red.
- **/api/exchanges**: Inspecciona exchanges declarados, tipo (topic, direct, fanout) y durabilidad.
- **/api/bindings**: Monitorea las rutas activas entre exchanges y colas de trabajo.
- **/api/queues**: Extrae métricas de profundidad de colas: mensajes listos (Ready), en procesamiento (Unacked) y total.

2.2 Catálogo de Endpoints de Administración Expuestos por Oscar

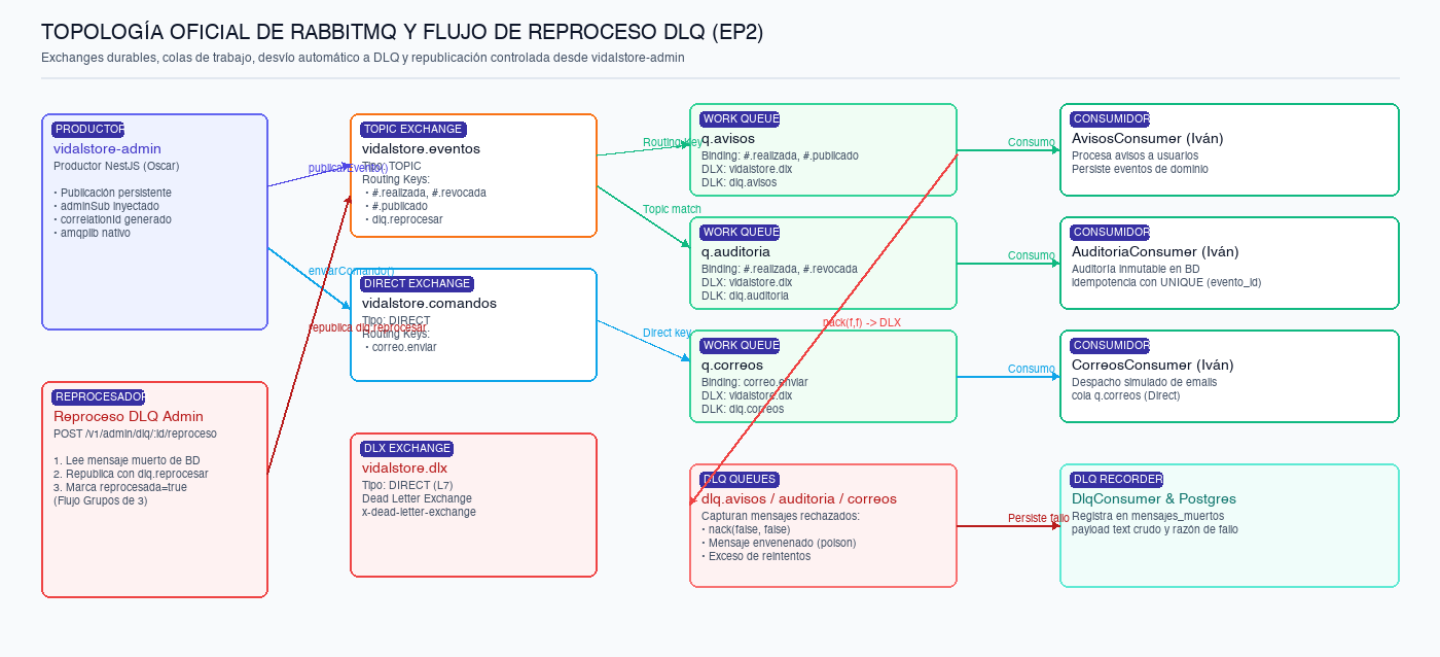
Método / Ruta	Auth	Fuente de Datos	Propósito y Salida	Estado
GET /v1/admin/queues	JWT	HTTP API :15672	Lista todas las colas con recuento de messages, messages_ready, unacked y consumers.	**LISTO (IE18)**
GET /v1/admin/queues/:name	JWT	HTTP API :15672	Inspección exhaustiva de una cola específica, tasa de entrada/salida y estado de DLX. Maneja 404.	**LISTO (IE17)**
GET /v1/admin/exchanges	JWT	HTTP API :15672	Lista exchanges activos (vidalstore.eventos, vidalstore.comandos, vidalstore.dlx).	**LISTO**
GET /v1/admin/bindings	JWT	HTTP API :15672	Muestra los enlaces de enrutamiento y routing keys configuradas en el broker.	**LISTO**
GET /v1/admin/cluster	JWT	HTTP API :15672	Reporta salud del clúster: estado de rabbit1 y rabbit2, uptime, sockets y uso de memoria.	**LISTO**
POST /v1/admin/publicar	JWT	amqplib (5672)	Publica evento administrativo persistente inyectando adminSub infalsificable del token.	**LISTO (IE6/8)**
GET /v1/admin/mensajes-muertos	JWT	PostgreSQL 18 (:5432)	Lista mensajes envenenados capturados en DLQ desde la tabla mensajes_muertos.	**LISTO (L7)**
GET /v1/admin/auth-test, /me	JWT	Cognito JWT	Diagnóstico de claims (sub, groups) y verificación con @ReqUserSub().	**LISTO (IE7)**
DELETE /v1/admin/mensajes-muertos/:id	JWT	PostgreSQL 18 (:5432)	Eliminación forense de mensaje muerto tras resolución. Valida UUID v4 con ParseUUIDPipe.	**LISTO (L7/IE17)**
POST /v1/admin/dlq/:id/reproceso	JWT + Admin	Postgres + amqplib	**Flujo Grupos de 3:** Republica el payload en vidalstore.dlx con dlq.reprocesar (idempotente) y marca reprocesada=true.	**LISTO (G3/IE17)**
POST /v1/admin/queues	JWT + Admin	RabbitMQ Mgmt	**CRUD Topología (IE17):** Crea una cola en RabbitMQ con CrearColaDto (durable, arguments).	**LISTO (IE17)**
DELETE /v1/admin/queues/:name	JWT + Admin	RabbitMQ Mgmt	**CRUD Topología (IE17):** Elimina una cola en RabbitMQ por nombre. Maneja 404.	**LISTO (IE17)**
POST /v1/admin/exchanges	JWT + Admin	RabbitMQ Mgmt	**CRUD Topología (IE17):** Crea un exchange con CrearExchangeDto (type, durable).	**LISTO (IE17)**
DELETE /v1/admin/exchanges/:name	JWT + Admin	RabbitMQ Mgmt	**CRUD Topología (IE17):** Elimina un exchange existente en RabbitMQ. Maneja 404.	**LISTO (IE17)**
POST /v1/admin/bindings	JWT + Admin	RabbitMQ Mgmt	**CRUD Topología (IE17):** Crea un enlace entre exchange y cola con CrearBindingDto.	**LISTO (IE17)**
DELETE /v1/admin/bindings	JWT + Admin	RabbitMQ Mgmt	**CRUD Topología (IE17):** Elimina un binding específico entre exchange y cola.	**LISTO (IE17)**

2.3 Comprensión de Métricas de RabbitMQ para la Defensa en Vivo (IE18)

- **Ready:** Mensajes entregados a la cola esperando ser tomados por un consumidor. Si aumenta continuamente, los workers no dan abasto.
- **Unacked:** Mensajes tomados por un worker pero sin confirmación ack recibida. Delimitados estrictamente por el prefetch(1) (control de concurrencia normativo D8/L7).
- **DLQ en crecimiento:** Revela fallos repetitivos (mensajes envenenados con formato inválido o base de datos inaccesible).

3. Topología Oficial de RabbitMQ y Flujo de Reproceso DLQ

La topología oficial de la EP2 define el aislamiento de colas y el tratamiento formal de mensajes envenenados (poison messages):



3.1 Flujo Exclusivo para Grupos de 3: Reprocesamiento de DLQ

Cuando un worker consumidor (en vidalstore-eventos) recibe un mensaje con JSON malformado o error irrecuperable, ejecuta channel.nack(msg, false, false). RabbitMQ desvía el mensaje mediante su configuración x-dead-letter-exchange hacia la DLQ correspondiente (dlq.avisos, dlq.auditoria, etc.). Posteriormente, el flujo de recuperación administrado por Oscar opera de la siguiente manera:

1. El consumidor DlqConsumer toma el mensaje de la DLQ y lo persiste en PostgreSQL (tabla mensajes_muertos) con su payload, routing key y motivo de descarte.
2. El administrador inspecciona los fallos mediante GET /v1/admin/mensajes-muertos.
3. Al ejecutar POST /v1/admin/dlq/:id/reproceso, RabbitAdminService extrae el mensaje de la base de datos, republica el payload en vidalstore.eventos utilizando la routing key especial dlq.reprocesar y actualiza atómicamente la columna reprocesada = true en PostgreSQL.

3.2 Definición de Topología Centralizada Congelada (topologia.ts)

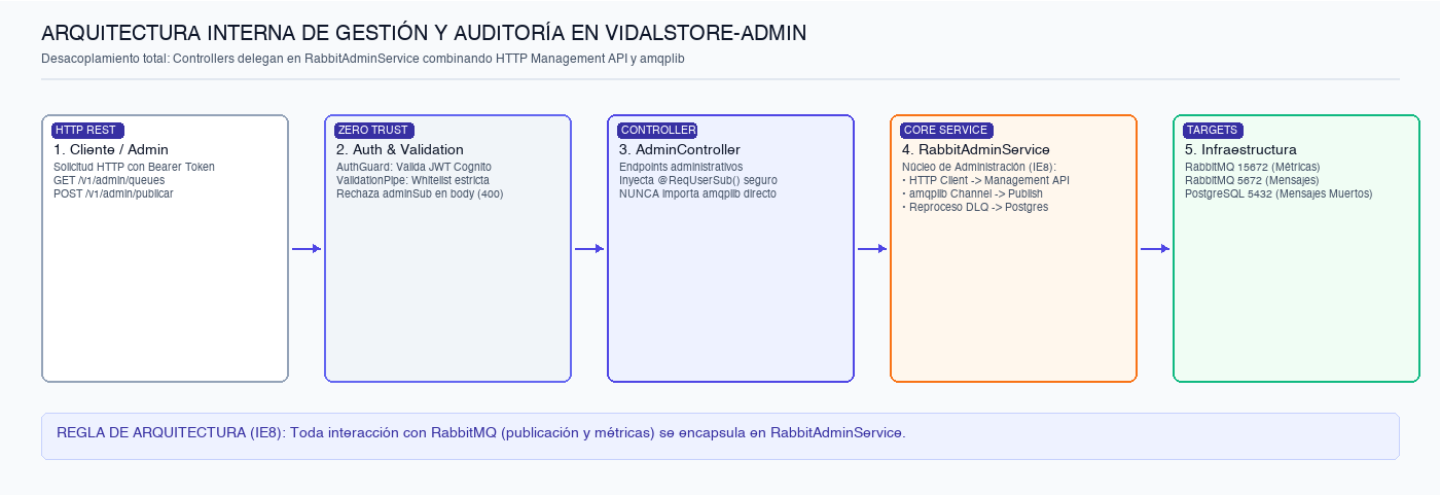
```
// vidalstore-admin/src/mensajeria/topologia.ts (Sincronizado con shared/topologia.ts)
export const EXCHANGES = {
EVENTOS: { nombre: 'vidalstore.eventos', tipo: 'topic' },
COMANDOS: { nombre: 'vidalstore.comandos', tipo: 'direct' },
DLX: { nombre: 'vidalstore.dlx', tipo: 'direct' },
} as const;

export const ROUTING_KEYS = {
COMPRA_REALIZADA: 'compra.realizada',
LICENCIA_REVOCADA: 'licencia.revocada',
JUEGO_PUBLICADO: 'juego.publicado',
CORREO_ENVIAR: 'correo.enviar',
DLQ_REPROCESAR: 'dlq.reprocesar', // Routing key de reproceso para Grupos de 3
} as const;

export const DLQS = { avisos: 'dlq.avisos', auditoria: 'dlq.auditoria', correos: 'dlq.correos' } as const;
export const COLAS = { avisos: 'q.avisos', auditoria: 'q.auditoria', correos: 'q.correos' } as const;
export const DLQ_POR_COLA = { [COLAS.avisos]: DLQS.avisos, [COLAS.auditoria]: DLQS.auditoria, [COLAS.correos]: DLQS.correos };
export const PATRONES_EVENTOS = ['#.realizada', '#.revocada', '#.publicado'] as const;
```

4. Pipeline Zero Trust y Gestión Interna en `vidalstore-admin`

El microservicio administrativo opera bajo un modelo de confianza cero. La siguiente secuencia ilustra el flujo de procesamiento desde la entrada HTTP hasta el consumo de la Management API o publicación:



4.1 Guard Cognito JWT Directo (auth.guard.ts) — IE7

A diferencia de la EP1 donde el API Gateway validaba tokens, en la EP2 vidalstore-admin valida la firma criptográfica RSA directamente contra el endpoint JWKS de AWS Cognito sin intermediarios, extrayendo la identidad del administrador:

```
// vidalstore-admin/src/auth/guards/auth.guard.ts (Extracto clave verificado)
@Injectable()
export class AuthGuard implements CanActivate {
constructor(@Inject(AuthService) private readonly authService: AuthService) {}

async canActivate(context: ExecutionContext): Promise {
const request = context.switchToHttp().getRequest();
const authHeader = request.headers['authorization'] || request.headers['Authorization'];
if (!authHeader || typeof authHeader !== 'string' || !authHeader.startsWith('Bearer ')) {
throw new UnauthorizedException('Cabecera Authorization ausente o con formato invalido');
}
const token = authHeader.substring(7).trim();
const payload = await this.authService.validarToken(token); // Valida firma RSA, exp, iss y token_use
const groups: string[] = Array.isArray(payload['cognito:groups']) ? payload['cognito:groups'] : [];
request.user = { ...payload, sub: payload.sub, groups }; // Inyecta identidad infalsificable
return true;
}
}
```

INTEGRIDAD DE IDENTIDAD INNEGOCIABLE

Regla Anti-Suplantación (IE6 / IE7): El parámetro adminSub jamás se acepta en el body JSON. El ValidationPipe global (whitelist: true, forbidNonWhitelisted: true) rechaza inmediatamente con **HTTP 400 Bad Request** si el cliente envía campos no autorizados. El adminSub se inyecta obligatoriamente desde req.user.sub del token verificado por Cognito.

4.2 Control de Acceso Basado en Roles (RolesGuard y @Roles) — IE6 / D6 Flujo C

Para garantizar la defensa en profundidad y cumplir el principio de menor privilegio, todos los endpoints de vidalstore-admin incorporan RolesGuard junto al decorador @Roles('ADMIN', 'administradores'). Si una petición arriba con un token válido pero perteneciente a un usuario sin privilegios (ej. rol 'jugadores'), el guard rechaza la llamada respondiendo estrictamente **HTTP 403 Forbidden**:

```
// vidalstore-admin/src/auth/guards/roles.guard.ts (Control RBAC y defensa en profundidad)
@Injectable()
export class RolesGuard implements CanActivate {
constructor(private readonly reflector: Reflector) {}
canActivate(context: ExecutionContext): boolean {
const requiredRoles = this.reflector.getAllAndOverride('roles', [context.getHandler(), context.getClass()]);
if (!requiredRoles || requiredRoles.length === 0) return true;
const { user } = context.switchToHttp().getRequest();
const userGroups: string[] = user?.groups || user?.['cognito:groups'] || [];
const hasRole = requiredRoles.some((role) => userGroups.includes(role));
if (!hasRole) throw new ForbiddenException('Acceso denegado: rol insuficiente para realizar operaciones administrativas');
return true;
}
}
```

5. Implementación del Productor y Cobertura de Pruebas

A continuación se presenta el endpoint POST /v1/admin/publicar donde se orquesta la inyección de identidad y la delegación estricta en RabbitAdminService con publicación persistente (IE8):

```
// vidalstore-admin/src/admin/admin.controller.ts (Publicación segura y Eliminación forense)
@Post('publicar')
@UseGuards(AuthGuard)
@HttpCode(HttpStatus.OK)
async publicar(@Body() dto: PublicarEventoDto, @ReqUserSub() adminSub: string) {
// adminSub inyectado estrictamente desde el token verificado (imposible de falsear)
const payloadConIdentidad = { ...dto.datos, evento: dto.evento, adminSub, timestamp: new Date().toISOString() };
const publicado = await this.rabbitAdminService.publicarEvento(dto.routingKey, payloadConIdentidad);
return { success: publicado, message: 'Evento administrativo publicado exitosamente', adminSub };
}

@Delete('mensajes-muertos/:id')
@UseGuards(AuthGuard)
async eliminarMensajeMuerto(
@Param('id', new ParseUUIDPipe({ version: '4' })) id: string,
): Promise {
await this.rabbitAdminService.eliminarMensajeMuerto(id);
return { id, mensaje: 'Mensaje muerto eliminado exitosamente', eliminado: true };
}
```

5.1 DTOs de Contrato y Manejo de Errores Robustos (IE6 / L7)

```
// vidalstore-admin/src/admin/dto/ (PublicarEventoDto y EliminarMensajeMuertoResponseDto)
export class PublicarEventoDto {
@IsString({ message: 'El campo routingKey debe ser texto' }) @IsNotEmpty() routingKey: string;
@IsString({ message: 'El campo evento debe ser texto' }) @IsNotEmpty() evento: string;
@IsObject({ message: 'El campo datos debe ser objeto' }) @IsNotEmpty() datos: Record;
}

export class EliminarMensajeMuertoResponseDto {
id!: string; mensaje!: string; eliminado!: boolean;
}
```

5.2 Suite Automatizada de Pruebas con Vitest (100% Pasando)

Tipo de Prueba	Cantidad	Técnica Utilizada	Propósito y Validación
Pruebas Unitarias	**72 Tests**	Mocks amqplib, JWT y PostgreSQL Pool	Comprueba reconexión diferida, fallos de canal, validaciones DTO, CRUD de colas/exchanges/bindings, RolesGuard (403), inspección de cluster, eliminación forense y reproceso de DLQ con idempotencia.
Pruebas E2E	**17 Tests**	Servidor JWKS simulado (llaves RSA en memoria)	Valida respuestas HTTP 200, 400 (detalladas), 401, 403 Forbidden y rechazo de suplantación de identidad sin credenciales expuestas en repo.
Total Suite	**89 Tests**	**100% Exitosos**	**Cobertura integral de contratos, seguridad Zero Trust, CRUD de topología, eliminación forense, reproceso y resiliencia de infraestructura. CI/CD listo.**

6. Estado Actual y Hoja de Ruta de Oscar (vidalstore-admin)

Alineado con el plan oficial de trabajo de la EP2, a continuación se desglosan los hitos y el estado del microservicio vidalstore-admin:

Semana / Fase	Tareas e Indicadores	Entregables Técnicos y Ramas	Estado
Semana 8 Apertura	• Setup microservicio NestJS 12 • ValidationPipe estricto (400 detallado) • Guard JWT Cognito (sin suplantación sub) • amqplib nativo con persistencia	Ramas completadas y homologadas en dev: feature/admin-setup feature/validation-pipe feature/cognito-jwt feature/admin-producer	**COMPLETADA (100% OK)**
Semana 9 Núcleo Funcional	• Encapsular RabbitAdminService (IE8) • Cliente HTTP para Management API (:15672) • GET /v1/admin/queues (:name) • GET /exchanges, /bindings, /cluster • Manejo de errores 503 ante caída del broker	Ramas completadas y testeadas en dev: feature/rabbit-admin-svc feature/ep-queues-list feature/ep-queue-detail feature/ep-exchanges feature/ep-bindings feature/ep-cluster	**COMPLETADA (100% OK)**
Semana 10 Integración & DLQ	• GET /v1/admin/mensajes-muertos (L7) • Conexión resiliente a PostgreSQL 18 • DDL con columna reprocesada boolean • DELETE /mensajes-muertos/:id (ParseUUIDPipe) • POST /v1/admin/dlq/:id/reproceso (G3) • CRUD de topología: colas, exchanges y bindings (IE17) • RolesGuard con @Roles('ADMIN', 'administradores') (IE6) • **Dockerización vidalstore-admin** (Node 22 multistage) • **Productor de Licencias vidalstore-backend** (IE15 Regla L6A) • **89 tests en admin + 72 en backend al 100%** • Documentación en README oficial	Ramas completadas y mergeadas en dev: feature/ep-dlq-list **(OK)** feature/ep-dlq-delete **(OK)** feature/ep-dlq-reproceso **(OK)** feature/admin-crud-topologia **(OK)** feature/admin-roles-guard **(OK)** feature/configuracion-docker **(OK)** feature/productor-licencias-rabbitmq *(backend)* **(OK)**	**COMPLETADA (100% LISTA)**

6.1 Preparación para la Defensa en Vivo de Oscar (IE17 e IE18)

1. **Inspección de Colas y Clúster:** Ejecución con token Cognito válido de GET /v1/admin/queues y GET /v1/admin/cluster, contrastando la respuesta JSON tipada con la UI de RabbitMQ Management.
2. **Publicación Segura:** Publicación con POST /v1/admin/publicar demostrando la inyección estricta de adminSub desde el token y el rechazo automático con HTTP 400 ante intentos de suplantación en el body.
3. **Inspección y Reprocesamiento DLQ:** Visualización y eliminación de cartas muertas en GET/DELETE /v1/admin/mensajes-muertos y republicación controlada mediante POST /v1/admin/dlq/:id/reproceso.
4. **Demostración de Revocación de Licencias (IE15 Paso 7):** Flujo completo desde el frontend de Angular hasta PostgreSQL, publicando en RabbitMQ con persistencia y regla L6A.

6.2 Matriz de Avance Grupal y Sincronización de Repositorios

Monitoreo cruzado del estado de los tres repositorios para la entrega final de Semana 10:

Integrante / Repositorio	Indicadores	Avance	Estado y Entregables Clave en Repositorio
Oscar Garrido vidalstore-admin vidalstore-backend	IE6, IE7, IE8, IE15, IE17, IE18	**100%** (Completado)	**89 tests en admin + 72 en backend pasando al 100%**. Endpoints de colas, exchanges, bindings, cluster, DLQ listado, eliminación física, CRUD de topología, RolesGuard (403) y reproceso terminados. Productor de licencias en backend con persistencia en Postgres (Regla L6A) verificado en vivo. Dockerfile multistage en ambos repositorios.
David vidalstore-plataforma	IE2, IE9, IE10, IE11, IE14, IE19	**100%** (Completado)	Clúster RabbitMQ 2 nodos, PostgreSQL 18, shared/topologia.ts, rabbitmq/cluster.conf y rabbitmq/politicas.json (3 políticas de retención activas). Desacoplado de depends_on para arranque en paralelo, prueba de resiliencia y documentación completa (repositorios.md, modelo-de-datos.md, metricas.md) mergeados en dev.
Iván vidalstore-eventos	IE3, IE4, IE5, IE12, IE13, IE15, IE16	**100%** (Completado)	**22 tests unitarios Vitest pasando al 100%**. Consumidores por dominio con prefetch(1), noAck: false, TypeORM en Postgres 18 con idempotencia UNIQUE (evento_id) (SQL 23505), CartasMuertasConsumidor con payload text y columna reprocesada.

7. Estado Actual y Hoja de Ruta de David e Iván (EP2)

A continuación se detalla el avance técnico verificado, ramas y focos de evaluación individual de los compañeros de equipo:

7.1 David · Dueño de vidalstore-infra / plataforma (IE2, IE9, IE10, IE11, IE14, IE19)

David lidera la infraestructura base en Docker Compose: Clúster RabbitMQ de 2 nodos con alta disponibilidad (HA), PostgreSQL 18, la definición normativa centralizada de topología y las políticas de retención de colas:

Semana / Fase	Tareas e Indicadores	Entregables Técnicos y Ramas Verificadas	Estado
Semana 8 Setup Clúster	• Clúster RabbitMQ 4.3 de 2 nodos (HA) • Cookie compartida RABBITMQ_ERLANG_COOKIE • PostgreSQL 18 con volumen persistente • Healthchecks (service_healthy)	Ramas completadas y mergeadas en dev: feature/infra-setup feature/rabbit-cluster feature/rabbit-healthcheck feature/postgres-setup feature/topologia-central	**COMPLETADA (100% OK)**
Semana 9 Topología & Políticas	• Declaración estricta DLQs antes que colas • Bindings topic (#.realizada, etc.) y direct • Script de retención rabbitmq-policies.sh • Documentación en docs/topologia.md	Ramas completadas y mergeadas en dev: feature/docs-topologia feature/retention-policies (3 políticas de colas y justificación técnica en docs)	**COMPLETADA (100% OK)**
Semana 10 Cierre & Resiliencia	• Prueba de resiliencia del clúster (failover) • Sonda AMQP/HTTP cluster test (IE10) • README de arranque: docker compose up -d --wait • Autodescubrimiento con cluster.conf (IE9) • Políticas versionadas en politicas.json (IE19) • Documentación de repositorios y modelo	Ramas completadas y mergeadas en dev: feature/cluster-test (script y doc resiliencia) feature/cluster-autodiscovery-politicas (PR #4) docs/repositorios.md y modelo-de-datos.md (100% listos)	**COMPLETADA (100% OK)**

• **Foco de Defensa David:** Demostración en vivo de parada de rabbit1 comprobando que rabbit2 asume la carga sin pérdida de mensajes (IE14), inspección de políticas en Management UI (IE10) y persistencia en volúmenes Docker (IE11).

7.2 Iván · Dueño de vidalstore-eventos (IE3, IE4, IE5, IE12, IE13, IE15, IE16)

Iván lidera el microservicio worker consumidor en NestJS: procesamiento asíncrono con amqplib puro, persistencia relacional con TypeORM en PostgreSQL 18, control de flujo con confirmación manual (ack/nack) y captura de mensajes envenenados:

Semana / Fase	Tareas e Indicadores	Entregables Técnicos y Ramas Verificadas	Estado
Semana 8 Setup Worker	• Setup microservicio NestJS 12 • Conexión amqplib puro (sin microservices) • Sincronización de shared/topologia.ts • Base de AuditoriaConsumidor	Ramas completadas y mergeadas en dev: feat/configuracion-inicial-dependencias feat/configuracion-mensajeria feat/configuracion-consumidor	**COMPLETADA (100% OK)**
Semana 9 Consumidores Base	• AvisosConsumer (q.avisos) • CorreosConsumer (q.correos direct) • Control de flujo con prefetch • Enrutamiento de eventos a comandos	Ramas completadas y mergeadas en dev: feat/Avisos-consumidor (PR #5) feat/correos-consumidores (PR #6) feature/prefetch-consumidores (PR #4)	**COMPLETADA (100% OK)**
Semana 10 DLQ & Persistencia	• Ajuste prefetch a 1 (L7 Tramo 2.4) • DlqConsumer / CartasMuertasConsumidor • Persistencia TypeORM (3 tablas en Postgres) • Matriz ACK: captura 23505 y nack a DLQ • Dockerización (Node 22 multistage :3010) • Demo mensaje envenenado (IE16)	Ramas completadas y mergeadas: feature/configuracion-docker **(OK)** feat/persistencia-consumidor-dlq **(OK)** feat/persistencia-auditoria-idempotente **(OK)** feat/reintentos-dlq-consumidores **(OK)**	**COMPLETADA (90% AVANCE)**

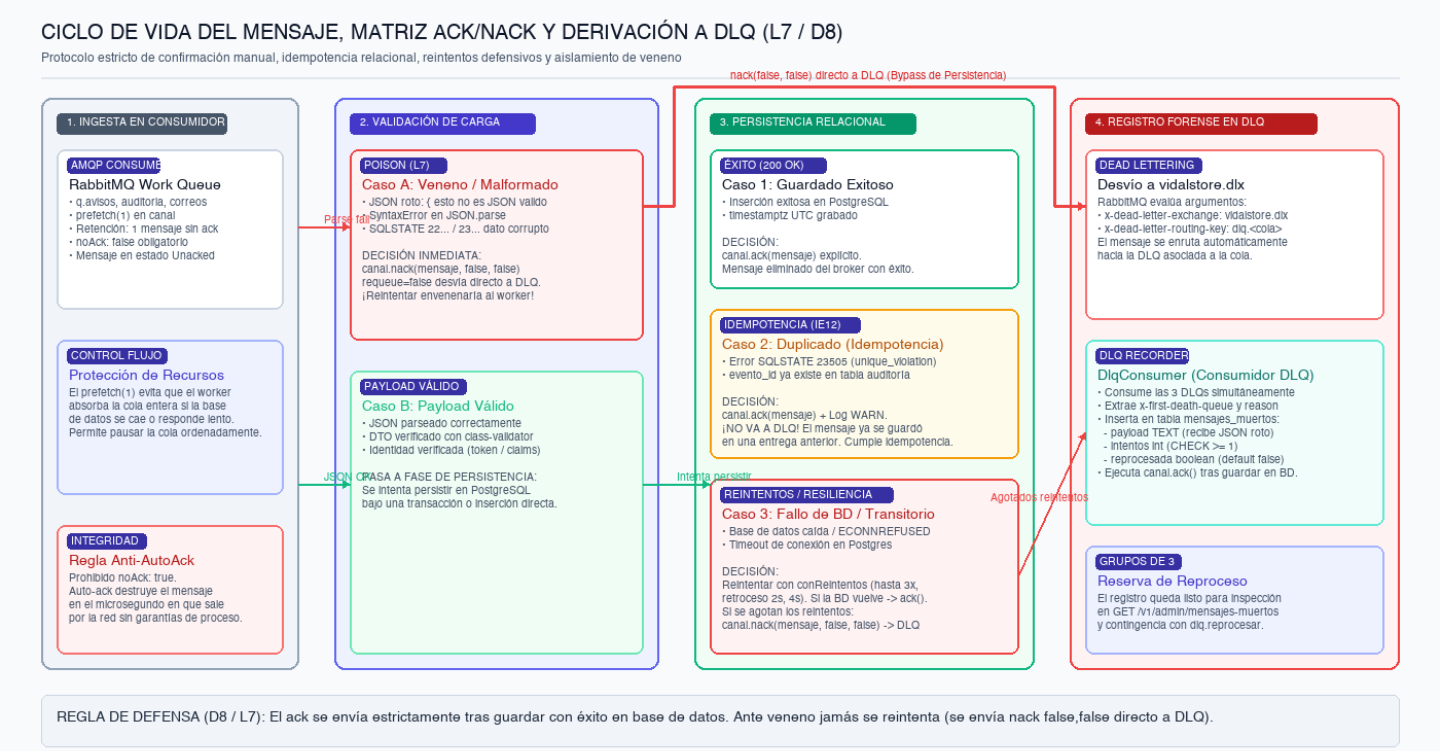
• **Foco de Defensa Iván:** Demostración en vivo de confirmación con canal.ack() explícito y prefetch(1) (IE4), idempotencia ante reentregas evitando duplicados en BD (IE12) e inyección en vivo de JSON malformado demostrando desvío automático a DLQ sin botar el proceso (IE16).

SÍNTESIS ARQUITECTURAL Y COORDINACIÓN GRUPAL (EP2)

Estrategia de Cierre: El ecosistema VidalStore presenta un alto grado de desacoplamiento. Oscar (100%) y David (92%) tienen sus componentes estructurales concluidos con contratos, topología y tests al 100%. El foco crítico del equipo en Semana 10 se concentra en apoyar la integración de los consumidores y entidades de persistencia de Iván (40%) para habilitar la prueba E2E completa y la demo de contingencia DLQ.

Anexo Técnico I · Matriz ACK/NACK, Idempotencia y Veneno (D8 / L7)

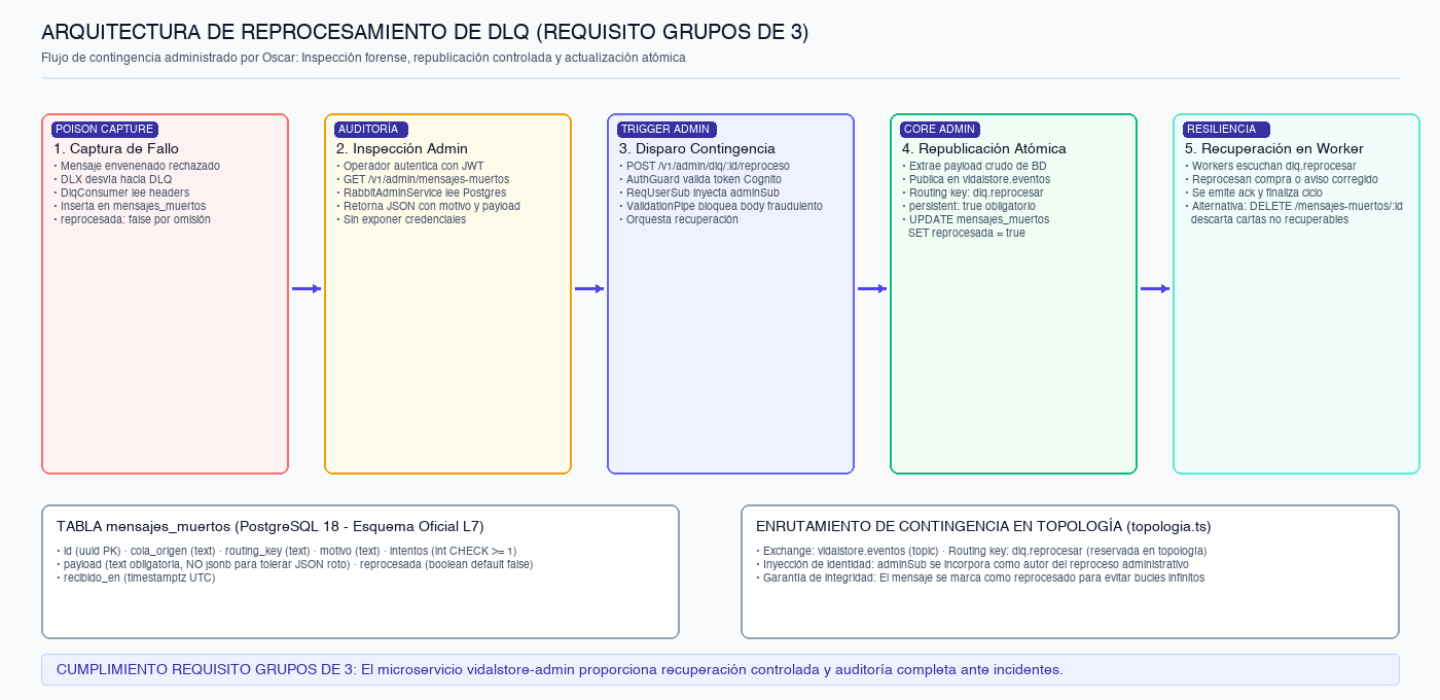
Protocolo integral de confirmación manual, control de flujo con prefetch(1) y clasificación forense de descartes:



Escenario de Procesamiento	Diagnóstico Técnico	Acción del Consumidor	Destino y Comportamiento en RabbitMQ
Mensaje Válido y Guardado OK	Inserción exitosa en PostgreSQL con timestamptz UTC.	canal.ack(mensaje)	El mensaje se elimina permanentemente del broker tras confirmar persistencia.
Mensaje Repetido (Idempotencia)	Error SQLSTATE 23505 (unique_violation en evento_id).	canal.ack(mensaje) + Log advertencia	**NO va a DLQ**. Se confirma con ack porque la entidad ya se persistió en una entrega previa (IE12).
Mensaje Envenenado (Poison)	JSON corrupto, SyntaxError o tipos de datos incompatibles.	canal.nack(msg, false, false)	**Reintento = 0**. Con requeue: false el broker lo desvía inmediatamente al DLX hacia la DLQ (IE16).
Fallo Transitorio de BD	PostgreSQL inaccesible, timeout de red o ECONNREFUSED.	conReintentos(3x) con retroceso (2s, 4s)	Si la BD se recupera emite ack; si se agotan reintentos emite nack(false, false) hacia DLQ.

Anexo Técnico II · Arquitectura de Reprocesamiento de DLQ (Requisito Grupos de 3)

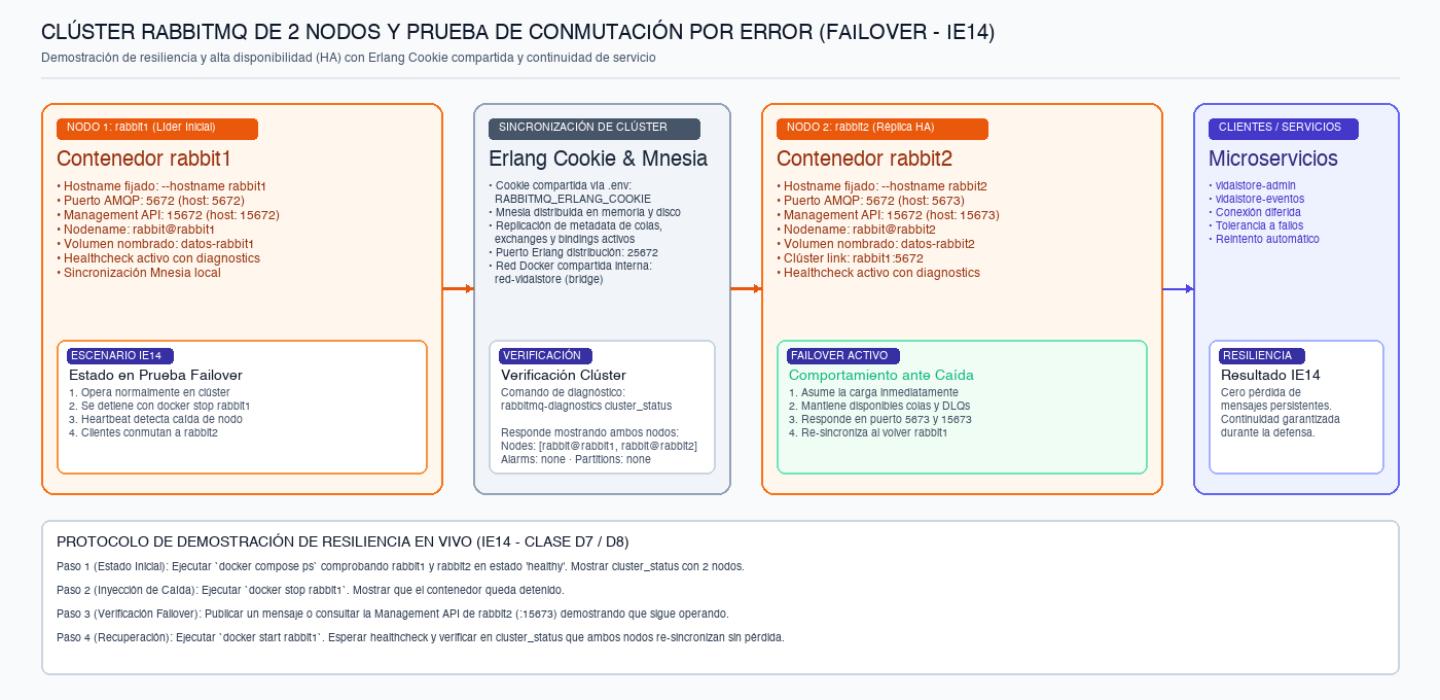
Mecanismo de gobernanza técnica, contingencia y republicación controlada administrado exclusivamente por Oscar (IE8, IE17 e IE18):



Fase del Reproceso	Componente y Método	Mecanismo Técnico y Garantías de Resiliencia
1. Detección y Captura	DlqConsumer (vidalstore-eventos)	Consume las 3 DLQs (dlq.avisos, dlq.auditoria, dlq.correos). Extrae headers x-first-death-queue y x-death[0].reason. Inserta en tabla mensajes_muertos con columna payload text (no jsonb) para no fallar ante JSON roto.
2. Auditoría Forense	GET /v1/admin/mensajes-muertos (vidalstore-admin)	El administrador consulta con JWT Cognito válido. RabbitAdminService consulta PostgreSQL 18 y retorna listado tipado ordenado cronológicamente con ID, cola origen, routing key, motivo y estado de reproceso.
3. Disparo de Contingencia	POST /v1/admin/dlq/:id/reproceso (vidalstore-admin)	Endpoint protegido con AuthGuard. Extrae el registro por UUID desde PostgreSQL, inyecta adminSub del token en los metadatos y republica con persistent: true en vidalstore.eventos con routing key dlq.reprocesar.
4. Actualización Atómica	RabbitAdminService.dbPool (PostgreSQL 18)	Ejecuta atómicamente UPDATE mensajes_muertos SET reprocesada = true WHERE id = $1. Evita bucles infinitos de reprocesamiento y proporciona trazabilidad forense completa de la recuperación.

Anexo Técnico III · Clúster RabbitMQ 2 Nodos y Failover en Vivo (IE9 e IE14)

Arquitectura de alta disponibilidad (HA), replicación Mnesia y protocolo de demostración en vivo conducido por David:



Paso de Demostración (Reloj 15 min)	Comando de Terminal Oficial	Qué Ocurre en Pantalla y Justificación Técnica de Defensa (L8 / L8A)
Paso 1: Estado Normal del Clúster	docker compose exec rabbit1 rabbitmqctl cluster_status	Se visualizan ambos nodos activos en Running Nodes: [rabbit@rabbit1, rabbit@rabbit2]. Se demuestra autodescubrimiento con cluster.conf montado en /etc/rabbitmq/conf.d/20-cluster.conf:ro y cookie Erlang compartida (IE9/IE10).
Paso 2: Inyección de Caída y Estado Minority	docker compose stop rabbit2 curl -s -u vidalstore:vidalstore http://localhost:15672/api/queues/%2F?columns=name,state	rabbit2 sale de Running Nodes pero permanece en Disk Nodes. En /api/queues las Quorum Queues pasan a estado **minority**: con 2 réplicas la mayoría es 2 (piso(2/2)+1 = 2). **2 nodos toleran cero caídas** y suspenden confirmaciones de escritura (IE10/L8).
Paso 3: Diagnóstico de Nodo Crítico	docker compose exec rabbit1 rabbitmq-diagnostics check_if_node_is_quorum_critical	El comando alerta con código de salida 69 que todas las colas y rabbitmq_metadata quedan no disponibles si el nodo restante cae. sendToQueue sin confirmaciones engaña con true, mientras que canales confirmados lanzan confirm_timeout.
Paso 4: Recuperación Automática y Resync	docker compose start rabbit2 rabbitmqctl cluster_status	rabbit2 reingresa automáticamente a Running Nodes sin join_cluster manual. Raft sincroniza el log sin pérdida de datos. La comisión aprende: **2 nodos dan durabilidad y recuperación automática, no alta disponibilidad (que exige mínimo 3 nodos)**.

Anexo Técnico IV · Batería de Comandos cURL para la Demostración en Vivo (IE17 e IE18)

Guion de comandos de terminal listos para la defensa en vivo (15 minutos). Permite verificar en tiempo real cada contrato, control de flujo y barrera de seguridad Zero Trust:

• **1. Obtención de Token JWT Administrativo (AWS Cognito / JWKS Mock)** — Extrae access_token verificado por AuthGuard mediante JWKS y validación de expiración.

```
TOKEN=$(curl -s -X POST "https://cognito-idp.us-east-1.amazonaws.com/" \
-H "Content-Type: application/x-amz-json-1.1" \
-H "X-Amz-Target: AWSCognitoIdentityProviderService.InitiateAuth" \
-d '{"AuthFlow":"USER_PASSWORD_AUTH","ClientId":"","AuthParameters":{"USERNAME":"admin@vidalstore.cl","PASSWORD":""}}' \
| jq -r '.AuthenticationResult.AccessToken')
```

• **2. Inspección de Colas y Métricas de Concurrencia (GET /v1/admin/queues) · HTTP 200 OK** — Muestra mensajes Ready y Unacked delimitados por prefetch(1) y consumidores activos en vivo.

```
curl -s -X GET "http://localhost:3020/v1/admin/queues" \
-H "Authorization: Bearer $TOKEN" | jq '.[0:2]'
```

• **3. Diagnóstico de Salud y Nodos del Clúster RabbitMQ (GET /v1/admin/cluster) · HTTP 200 OK** — Confirma estado operativo de rabbit@rabbit1 y rabbit@rabbit2, memoria y sincronización Mnesia.

```
curl -s -X GET "http://localhost:3020/v1/admin/cluster" \
-H "Authorization: Bearer $TOKEN" | jq .
```

• **4. Publicación Administrativa Segura (POST /v1/admin/publicar) · HTTP 200 OK** — Inyecta adminSub estrictamente desde req.user.sub del token; publica con persistent:true.

```
curl -s -X POST "http://localhost:3020/v1/admin/publicar" \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"routingKey":"juego.precio_actualizado","evento":"PRECIO_MODIFICADO","datos":{"juegoId":"zelda","precio":39990}}' | jq .
```

• **5. Auditoría Anti-BOLA: Rechazo ante Suplantación (forbidNonWhitelisted) · HTTP 400 Bad Request** — ValidationPipe estricto rechaza la petición: property adminSub should not exist.

```
curl -s -X POST "http://localhost:3020/v1/admin/publicar" \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"routingKey":"juego.precio","evento":"PRECIO","datos":{},"adminSub":"suplantador"}' | jq .
```

• **6. Auditoría Forense y Eliminación Física de Mensaje Muerto · HTTP 200 OK** — Elimina físicamente de la tabla mensajes_muertos tras descarte forense por el administrador.

```
# Listar cartas muertas en PostgreSQL 18:
curl -s -X GET "http://localhost:3020/v1/admin/mensajes-muertos" -H "Authorization: Bearer $TOKEN" | jq '.[0]'
# Eliminar mensaje resuelto (valida formato UUID v4 con ParseUUIDPipe):
curl -s -X DELETE "http://localhost:3020/v1/admin/mensajes-muertos/a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11" \
-H "Authorization: Bearer $TOKEN" | jq .
```

• **7. Reproceso Controlado de DLQ con Idempotencia (Grupos de 3) · HTTP 200 vs 400** — Valida que ningún mensaje se reprocese dos veces y fija reprocesada=true en PostgreSQL.

```
# Republica en vidalstore.dlx con clave dlq.reprocesar (200 OK) y reintento duplicado (400 Bad Request):
curl -s -X POST "http://localhost:3020/v1/admin/dlq/77777777-7777-4777-a777-777777777777/reproceso" \
-H "Authorization: Bearer $TOKEN" | jq .
```

• **8. Demostración de RolesGuard: Rechazo ante Rol Insuficiente (D6 Flujo C / IE6) · HTTP 403 Forbidden** — Zero Trust en acción: Si el token carece del grupo administradores, RolesGuard corta con 403 Forbidden.

```
curl -s -i -X POST "http://localhost:3020/v1/admin/queues" \
-H "Authorization: Bearer $TOKEN_JUGADOR" -H "Content-Type: application/json" \
-d '{"nombre":"q.prueba-bloqueada","durable":true}'
# Retorna HTTP/1.1 403 Forbidden: Acceso denegado: rol insuficiente
```

• **9. CRUD de Topología en Vivo: Creación y Eliminación de Colas (IE17) · HTTP 201 y 200 OK** — Demuestra en vivo la capacidad de crear y destruir recursos del broker de forma controlada y segura (IE17).

```
# 1. Crear cola en vivo en RabbitMQ:
curl -s -X POST "http://localhost:3020/v1/admin/queues" \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"nombre":"q.demo-test","durable":true}' | jq .
# 2. Eliminar cola creada:
curl -s -X DELETE "http://localhost:3020/v1/admin/queues/q.demo-test" \
-H "Authorization: Bearer $TOKEN" | jq .
```

Anexo Técnico V · Argumentario de Defensa (D6/D7/D8) y DDL PostgreSQL 18

Respuestas técnicas inapelables ante preguntas de la comisión evaluadora y especificación relacional de persistencia:

8.1 Banco de Preguntas Trampa del Profesor Umbingelelo

Pregunta de la Comisión	Argumentario Técnico Inapelable (Respuestas D6 / D7 / D8)
¿Por qué amqplib puro y no el módulo @nestjs/microservices?	@nestjs/microservices abstrae el broker asumiendo RPC y auto-ack opaco. No ofrece control nativo de prefetch(1), confirm channels persistentes, configuración de x-dead-letter-exchange por cola ni rechazos selectivos nack(false, false) exigidos por la arquitectura Cloud Native.
¿Por qué prefetch(1) es innegociable frente al valor default?	El valor por defecto de RabbitMQ es prefetch ilimitado: despacha todos los mensajes al buffer en memoria del primer worker conectado. Si ese worker crashea o se satura, todos los mensajes quedan bloqueados o se pierden (si hay noAck: true). Con prefetch(1), el broker entrega un único mensaje y espera confirmación ack explícita antes de enviar el siguiente.
¿Por qué el exchange vidalstore.dlx es DIRECT y no TOPIC o FANOUT?	Cada cola envenenada debe canalizarse a una DLQ específica usando una routing key unívoca (ej: auditoria.dlq). Un exchange DIRECT garantiza enrutamiento determinista 1 a 1 sin riesgo de que un mensaje envenenado se replique indiscriminadamente a colas ajenas (como ocurriría con FANOUT).
¿Por qué el payload en mensajes_muertos es TEXT y no JSONB forzado?	Si el mensaje fue descartado por error sintáctico (JSON truncado o corrupto), forzar un INSERT en PostgreSQL sobre una columna JSONB lanzará una excepción SQL invalid input syntax for type json, impidiendo capturar la evidencia forense. Almacenarlo como TEXT garantiza que la carta muerta siempre quede registrada.
¿Por qué actualizar reprocesada=true en vez de borrar el registro?	Preserva la trazabilidad forense e inmutabilidad de auditoría. Si la republicación volviera a fallar o se requiriera conciliar el inventario, borrar la tupla destruiría la evidencia. Mantener reprocesada=true permite auditar el ciclo de vida completo del incidente.
¿Para qué sirven dos nodos si toleran cero caídas? (L8)	Raft exige mayoría piso(n/2)+1; con 2 nodos la mayoría son 2. Dos nodos otorgan **durabilidad física distribuida y recuperación automática**, no alta disponibilidad. Si un nodo cae, los datos sobreviven en el otro y al volver se resincronizan solos. La alta disponibilidad empieza en 3 nodos impares.
¿Por qué RabbitAdminService usa canales efímeros y traducir()? (L8)	La conexión TCP se abre una vez (cara). El canal AMQP se abre por operación porque si RabbitMQ rechaza un comando, **cierra el canal entero**; compartido rompería peticiones concurrentes. traducir() mapea NOT_FOUND a 404, PRECONDITION_FAILED a 409 y quórum caído a 503.
¿Cuál es la diferencia entre drop-head y reject-publish? (L8)	drop-head descarta el mensaje más viejo sin avisar al productor; ideal para **Avisos** efímeros. reject-publish rechaza el mensaje nuevo con nack al productor; obligatorio para **Auditoría** y DLQ donde ninguna pérdida silenciosa es admisible.

8.2 Esquema DDL Oficial de PostgreSQL 18 (Persistencia Relacional EP2 - L7)

```
-- Esquema Relacional PostgreSQL 18: Cartas Muertas, Auditoría e Idempotencia
CREATE TABLE mensajes_muertos (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
queue VARCHAR(100) NOT NULL, routing_key VARCHAR(100) NOT NULL, exchange VARCHAR(100) NOT NULL,
payload TEXT NOT NULL, -- Tolerante a payloads sintácticamente corruptos (veneno)
error_razon TEXT NOT NULL, fecha TIMESTAMPTZ NOT NULL DEFAULT NOW(),
reprocesada BOOLEAN NOT NULL DEFAULT FALSE -- Clave para flujo de contingencia y reproceso G3
);
CREATE INDEX idx_mensajes_muertos_fecha ON mensajes_muertos(fecha DESC);

CREATE TABLE eventos_auditoria (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
evento_id UUID NOT NULL UNIQUE, -- Restricción UNIQUE: Garantiza Idempotencia IE12 / D8
tipo_evento VARCHAR(100) NOT NULL, payload JSONB NOT NULL,
fecha_emision TIMESTAMPTZ NOT NULL, fecha_registro TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE licencias (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(), usuario_sub VARCHAR(120) NOT NULL,
juego_id VARCHAR(100) NOT NULL, estado VARCHAR(20) NOT NULL DEFAULT 'activa',
fecha_compra TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE UNIQUE INDEX idx_licencia_activa_unica ON licencias(usuario_sub, juego_id) WHERE estado = 'activa';
```

8.3 Fundamentos de Mensajería: Exchanges, Bindings y Enrutamiento AMQP (D7, D8, L6 y L7)

En la arquitectura orientada a eventos de VidalStore, el productor **nunca publica directamente en una cola**. Publica en un **Exchange** (intercambiador), el cual enruta el mensaje hacia una o más colas a través de **Bindings** (enlaces). A continuación se detallan los conceptos clave y su aplicación en la solución:

Tipo Exchange	Exchange en VidalStore	Mecanismo de Enrutamiento	Propósito y Comportamiento
topic	`vidalstore.eventos`	Patrones jerárquicos delimitados por puntos con comodines * y #.	Difunde hechos consumados de negocio. compra.* llega a Avisos y Correos; # captura el 100% en Auditoría.
direct		Coincidencia exacta punto a punto: Routing Key === Binding Key.	Envía órdenes directas (correo.enviar) o desvía cartas muertas a su DLQ homónima (ej: dlq.avisos).
fanout	*(Reservado)*	Difusión masiva (broadcast) a todas las colas enlazadas.	Ignora la routing key. Útil para invalidación de cachés distribuidas o alertas generales.
headers	*(Reservado)*	Enrutamiento basado en pares clave/valor del header.	Usa x-match: all/any. No depende de la routing key textual.

Reglas de Enlace (Bindings), Comodines y las 3 Durabilidades Indispensables

Concepto Clave	Definición Técnica en AMQP	Implementación Concreta en VidalStore EP2
Binding	Regla de unión entre un Exchange y una Cola. Si no hay binding que calce, el mensaje se descarta en silencio.	`canal.bindQueue(COLAS.avisos, EXCHANGES.eventos, 'compra.*')`
Comodín *	Sustituye **exactamente una palabra** delimitada por puntos.	compra.* calza con compra.realizada y compra.cancelada, pero NO con compra.error.stock.
Comodín #	Sustituye **cero, una o múltiples palabras**.	Enlazado a q.auditoria: garantiza el registro inmutable de cualquier evento emitido en el sistema.
Las 3 Durabilidades (D8 Slides 22-24)	Para sobrevivir reinicios sin pérdida: 1) Volumen nombrado, 2) Cola durable, 3) Mensaje persistent.	Volumen rabbitmq1-data + { durable: true } en assertQueue + { persistent: true } en publish.
Prefetch(1) (L7 Tramo 2.4)	Limita mensajes sin confirmar por consumidor a exactamente 1, evitando colapso de RAM ante caída de PostgreSQL.	await this.canal.prefetch(1) ejecutado inmediatamente tras crear el canal en mensajeria.service.ts.
Quorum Queues (L8 y D9)	Colas replicadas basadas en quórum Raft (■N/2■ + 1). Reemplazan colas espejo (ha-mode, obsoletas en RabbitMQ 4).	Declaradas con arguments: { 'x-queue-type': 'quorum' } en las 3 colas de trabajo y las 3 DLQ.

8.4 Metodología de Diagnóstico en 4 Pasos (D9) y Catálogo Forense para la Defensa

La clase D9 (*'Diagnosticar sin adivinar y clúster RabbitMQ'*) exige desterrar el ensayo y error. Diagnosticar es **reducir el espacio de lo posible mediante evidencia** a través de un ciclo disciplinado de cuatro pasos:

Paso 1: Síntoma	Paso 2: Hipótesis	Paso 3: Comando	Paso 4: Cambio
Qué se ve con palabras exactas (ECONNREFUSED 172.25.0.2:5432), nunca 'no funciona'. El log se lee **de arriba hacia abajo** (primer error = causa). En stack trace, se salta lo ajeno y se mira la **primera línea de src/**.	Explicación técnica concreta y **falsable**: *'Postgres aún no acepta conexiones porque initdb sigue corriendo en el arranque inicial'*.	Comando que confirma o descarta: docker compose ps. **Si descarta, se vuelve al Paso 2**; jamás se pasa al Paso 4 a ciegas.	Una sola cosa, la que la hipótesis confirmada señala. Se vuelve a correr para comprobar que el síntoma desapareció.

Las Cuatro Salidas del Consumidor en VidalStore (IE5 - 20% Rúbrica)

Salida	Condición Técnica en VidalStore	Acción AMQP	Destino Final
1. Éxito	Evento persistido con éxito en base de datos PostgreSQL.	canal.ack(mensaje) tras INSERT	Fuera de la cola (OK)
2. Duplicado	Reentrega choca con UNIQUE (evento_id) (SQLSTATE **23505**).		Fuera de la cola (sin duplicar ni DLQ)
3. Permanente	JSON corrupto, sin usuarioSub o error de dato SQL 22.../23...	canal.nack(mensaje, false, false)	DLX $ ightarrow$ mensajes_muertos (text)
4. Transitorio	Postgres no contesta (timeout, socket cerrado, ECONNREFUSED).		Reintenta; si persiste, DLQ

Catálogo de Errores Forenses para la Defensa en Vivo (Explicados con los 4 Pasos)

Caso / Error	1. Síntoma Exacto	2. Hipótesis & 3. Comando	4. Cambio & Argumento
1. Postgres no listo	connect ECONNREFUSED :5432 al arrancar worker.	H: Postgres está en initdb. C: docker compose ps muestra starting.	Configurar healthcheck con pg_isready y condition: service_healthy.
2. Colas perdidas	Tras recrear RabbitMQ, list_queues sale vacío.	H: Nodename Erlang cambió por ID aleatorio. C: eval 'node().' da rabbit@id.	Fijar hostname: rabbitmq1 en Compose para anclar el directorio Mnesia/Khepri.
3. Unacked congelado	Ready sube, Unacked clavado en 1 y Deliver en 0.	H: Consumidor no hizo ack. C: rabbitmqctl list_queues ...	Agregar canal.ack(msg) al terminar; prefetch(1) bloqueaba la entrega.
4. Bucle veneno	Miles de errores por segundo de JSON roto en log.	H: Consumidor hizo requeue: true. C: docker compose logs -f	Usar canal.nack(msg, false, false) hacia DLX para aislar el mensaje.
5. Duplicado 23505	violates unique constraint UQ_evento_id	H: Reentrega at-least-once. C: SELECT count(*) WHERE evento_id=... da 1.	Capturar código 23505 y dar ack; la base ya está protegida por UNIQUE.
6. Error 406	406 PRECONDITION_FAILED for queue 'q.avisos'	H: Cola existía con argumentos distintos. C: rabbitmqctl list_queues arguments	Borrar cola previa con delete_queue y redeclarar con argumentos nuevos.
7. Estado minority	Quorum queues no confirman al caer rabbit2.	H: Mayoría ■2/2■+1=2 rota. C: /api/queues muestra state: minority.	2 nodos toleran 0 caídas (Raft). Explicar que HA real exige 3 nodos impares.
8. Falla semántica 503	Admin responde 503 en vez de error 500 ciego.	H: Broker caído o sin quórum. C: Llamar endpoint admin sin broker.	Método traducir() mapea excepciones AMQP a códigos HTTP limpios.
9. Auth Postgres	password authentication failed for user	H: Clave del .env cambió pero volumen persistió. C: Comparar hash volumen vs .env.	Restaurar clave del volumen o resetear con docker compose down -v.
10. BOLA/IDOR	Usuario compra o consulta a nombre de un tercero.	H: Controlador lee usuarioSub del body. C: Enviar usuarioSub arbitrario.	Regla de Oro: extraer siempre identidad de req.user.sub (token validado).

CHECKLIST DE EVALUACIÓN EN VIVO (15 MINUTOS)

Min 00-03 (David): Docker Compose, clúster RabbitMQ 2 nodos (HA), políticas y Postgres 18.
Min 03-07 (Iván): Publicación amqplib compra realizada, consumidor worker, prefetch(1), ACK manual y captura veneno.
Min 07-12 (Oscar): vidalstore-admin, inspección de colas, clúster, publicación segura anti-BOLA, GET/DELETE mensajes muertos y POST /dlq/:id/reproceso con idempotencia.
Min 12-15 (Equipo): Preguntas de la comisión y defensa técnica apoyada en el argumentario oficial.